

Progettazione di applicazioni web full stack

Autenticazione e Autorizzazione

in NestJS

Sistema di pianificazione degli appelli

Lezione applicata al progetto d'esame
Basata sui materiali del corso del Prof. Devis Bianchini

Anno Accademico 2025/2026

Obiettivi della lezione

Al termine di questa lezione lo studente sarà in grado di:

- distinguere i concetti di *autenticazione* e *autorizzazione*;
- progettare la separazione delle responsabilità tra i moduli **users**, **auth** e **security**;
- implementare la strategia **passport-local** per la verifica delle credenziali;
- implementare la strategia **passport-jwt** per la validazione del token;
- proteggere le rotte mediante *guards* e decoratori personalizzati;
- realizzare il meccanismo dei ruoli (RBAC) applicato ai due attori **Docente** e **Segreteria**.

Indice

1	Inquadramento concettuale	3
1.1	Autenticazione vs Autorizzazione	3
1.2	Passport: la libreria di riferimento	3
1.3	Architettura a tre moduli	3
2	Preparazione dell'ambiente	5
2.1	Pacchetti npm da installare	5
2.2	Generazione delle librerie	5
2.3	Configurazione del file <code>.env</code>	5
3	Evoluzione della libreria <code>@server/users</code>	7
3.1	L'entità <code>UserEntity</code>	7
3.2	Il DTO <code>CreateUserDto</code>	7
3.3	Il servizio <code>ServerUsersService</code> : hashing della password	8
3.4	Il repository: persistenza dell'hash	8
4	La libreria <code>@server/auth</code>	10
4.1	Le interfacce	10
4.2	Il DTO di registrazione	10
4.3	Il servizio <code>ServerAuthService</code>	11
4.4	La strategia <code>LocalStrategy</code>	12
4.5	La guard <code>LocalAuthGuard</code>	13
4.6	La strategia <code>JwtStrategy</code>	13
4.7	Il controller <code>ServerAuthController</code>	14
4.8	Il modulo <code>ServerAuthModule</code>	15
4.9	File <code>index.ts</code> : cosa esportare e cosa no	15
5	La libreria <code>@server/security</code>	17
5.1	La guard <code>JwtAuthGuard</code>	17
5.2	La guard <code>RolesGuard</code> : il cuore dell'autorizzazione	17
5.3	Il decoratore <code>@Roles()</code>	18
5.4	Il decoratore <code>@CurrentUser()</code>	18
5.5	File <code>index.ts</code> di <code>security</code>	18
6	Applicazione al sistema di pianificazione appelli	20
6.1	Il ruolo come abilitazione	20
6.2	Esempio: il controller delle sessioni d'esame	20
6.3	Esempio: il controller degli appelli	21
6.4	Il servizio: il controllo «owner»	22
7	Configurazione Swagger con autenticazione	23
8	Flusso end-to-end di una richiesta	24
8.1	Registrazione di un nuovo docente	24
8.2	Richiesta protetta da <code>@Roles(SEGRETERIA)</code>	24
9	Esercizi di verifica	25
10	Riepilogo finale	26

1 Inquadramento concettuale

1.1 Autenticazione vs Autorizzazione

Prima di scrivere una sola riga di codice è fondamentale chiarire la distinzione tra due concetti spesso confusi:

Definizione

Autenticazione (*authentication*): è il processo che risponde alla domanda «**Chi sei?**». Verifica l'identità di un soggetto attraverso credenziali (e-mail e password) o un token che le sostituisce dopo il primo accesso.

Autorizzazione (*authorization*): è il processo che risponde alla domanda «**Cosa puoi fare?**». Una volta nota l'identità, decide se quel soggetto è abilitato a eseguire una specifica azione (per esempio, modificare un appello, configurare una sessione).

Nel nostro sistema:

- l'**autenticazione** stabilisce che chi accede è un certo utente registrato;
- l'**autorizzazione** decide se quell'utente, in quanto **DOCENTE** o **SEGRETERIA**, può eseguire l'azione richiesta.

1.2 Passport: la libreria di riferimento

NestJS non implementa direttamente la verifica delle credenziali; si appoggia a **Passport**, libreria standard de facto in ambiente Node.js.

Definizione

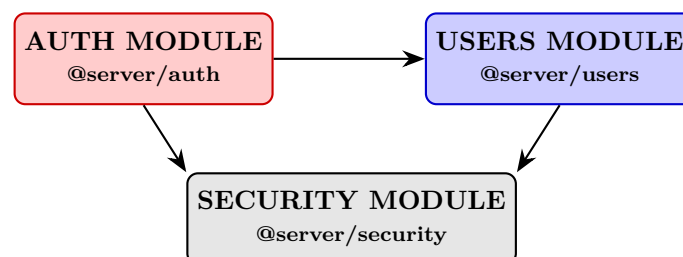
Passport è un *middleware* di autenticazione modulare: definisce un'interfaccia unica (*strategy*) e mette a disposizione decine di implementazioni concrete (local, JWT, OAuth, Google, GitHub...) tutte intercambiabili senza modificare la logica applicativa.

Useremo due strategie:

- **passport-local**: verifica e-mail e password in fase di *login*;
- **passport-jwt**: verifica un token *JSON Web Token* su tutte le richieste successive.

1.3 Architettura a tre moduli

Il sistema è suddiviso in tre librerie distinte. Questa separazione non è un dettaglio tecnico, ma una decisione progettuale che evita le **dipendenze circolari**.



Le tre librerie hanno questa funzione:

Libreria	Responsabilità
@server/users	Gestione anagrafica utenti: entità, repository, servizio CRUD, DTO. È la sola libreria che parla con il database degli utenti.
@server/auth	Logica di autenticazione: strategie Passport, controller di login e registrazione, generazione e firma del token JWT.
@server/security	Componenti riusabili da tutti i moduli protetti: JwtAuthGuard, RolesGuard, decoratori @Roles() e @CurrentUser(). <i>Non</i> dipende da auth , evitando il ciclo.

lightbulb Nota

Perché tre moduli e non uno solo? Se i *guards* stessero in @server/auth, qualsiasi libreria di dominio (per esempio @server/exam-planning) che vuole proteggere una rotta dovrebbe importare **auth**. Ma **auth** a sua volta potrebbe voler usare quelle libreria per logiche specifiche, generando un ciclo di dipendenze. La libreria **security**, neutrale, spezza il ciclo.

2 Preparazione dell'ambiente

2.1 Pacchetti npm da installare

Il primo passo è installare i pacchetti necessari. La tabella seguente riassume il ruolo di ognuno.

```
npm install @nestjs/passport passport passport-local
npm install @nestjs/jwt passport-jwt
npm install bcrypt
npm install -D @types/passport-jwt @types/bcrypt @types/passport-local
```

Pacchetto	Utilizzo
passport	Core engine di autenticazione
passport-local	Strategia per login con e-mail e password
passport-jwt	Strategia per autenticazione con token JWT
@nestjs/passport	Integrazione ufficiale di Passport in NestJS (guards, strategie)
@nestjs/jwt	Integrazione ufficiale per la firma e la verifica dei token
bcrypt	Hashing crittografico delle password con <i>salt</i>
@types/...	Definizioni TypeScript (solo in <i>dev dependencies</i>)

2.2 Generazione delle librerie

Le due nuove librerie si generano con i comandi Nx:

```
npx nx g @nx/nest:library libs/server/auth \
  --controller --service --importPath=@server/auth

npx nx g @nx/nest:library libs/server/security \
  --importPath=@server/security
```

lightbulb Nota

La libreria **security** è generata *senza* `-controller` e `-service` perché contiene solo guards e decoratori, non endpoint HTTP né business logic.

2.3 Configurazione del file .env

La firma dei token JWT richiede una chiave segreta. Si genera con un'unica riga di comando e si memorizza nel file `.env`:

```
node -e "console.log(require('crypto').randomBytes(16).toString('hex'))"
```

Il file `.env` dovrà contenere almeno:

```
PORT=3000
PG_HOST=127.0.0.1
PG_PORT=5432
PG_USERNAME=postgres
PG_PASSWORD=*****
PG_DATABASE=PFSPProject
SECRET_KEY=6165bae4141b2d3dd434ba20333b5433
```

Attenzione

Il file `.env` non va mai committato. Va aggiunto al `.gitignore`. La `SECRET_KEY` esposta pubblicamente compromette la sicurezza dell'intera applicazione, poiché chiunque può falsificare token JWT validi.

3 Evoluzione della libreria @server/users

La libreria utenti deve evolvere per ospitare due informazioni chiave: la **password cifrata** e il **ruolo**. Vediamo i tre file da modificare.

3.1 L'entità UserEntity

```
1 import { Entity, Column, PrimaryGeneratedColumn } from 'typeorm';
2
3 export enum UserRole {
4   DOCENTE = 'DOCENTE',
5   SEGRETERIA = 'SEGRETERIA',
6 }
7
8 @Entity('users')
9 export class UserEntity {
10   @PrimaryGeneratedColumn()
11   id: number;
12
13   @Column({ type: 'varchar', length: 255, nullable: false })
14   name: string;
15
16   @Column({ type: 'varchar', length: 320, nullable: false, unique: true })
17   email: string;
18
19   @Column()
20   passwordHash: string;
21
22   @Column({
23     type: 'enum',
24     enum: UserRole,
25     default: UserRole.DOCENTE,
26   })
27   role: UserRole;
28 }
```

Listing 1: libs/server/users/src/lib/user.entity.ts

Applicazione al progetto «Pianificazione Appelli»

Rispetto al template di partenza (che aveva USER e ADMIN), nel nostro sistema il ruolo riflette i due attori dell'UML: **Docente** (chi inserisce gli appelli) e **Segreteria** (chi configura le sessioni). Questa è l'unica modifica concettuale all'enum.

3.2 Il DTO CreateUserDto

I DTO (*Data Transfer Object*) definiscono lo schema dei dati in ingresso e in uscita dalle API. Combinati con il pacchetto `class-validator`, eseguono validazioni dichiarative.

```
npm install class-validator class-transformer
```

```
1 import { IsEmail, IsEnum, IsNotEmpty, IsString,
2       MinLength, Matches } from 'class-validator';
3 import { UserRole } from '../user.entity';
4
5 export class CreateUserDto {
6   @IsString()
7   @IsNotEmpty()
```

```
8   name: string;
9
10  @IsEmail()
11  @IsNotEmpty()
12  email: string;
13
14  @IsString()
15  @IsNotEmpty()
16  @MinLength(8)
17  @Matches(/[A-Z]/, {
18    message: 'La password deve contenere almeno una lettera maiuscola',
19  })
20  @Matches(/[?~!#@]/, {
21    message: 'La password deve contenere almeno un simbolo tra ? ~ ! # @',
22  })
23  password: string;
24
25  @IsEnum(UserRole, {
26    message: 'Ruolo valido: DOCENTE o SEGRETERIA',
27  })
28  role: UserRole;
29 }
```

Listing 2: libs/server/users/src/lib/dto/create-user.dto.ts

3.3 Il servizio ServerUsersService: hashing della password

La password **non viene mai memorizzata in chiaro**. Quando l'utente si registra, si calcola un *hash* crittografico tramite `bcrypt`, e nel database va soltanto quello.

```
1 import * as bcrypt from 'bcrypt';
2
3 async create(dto: CreateUserDto): Promise<UserEntity> {
4   const existing = dto.email
5     ? await this.usersRepository.findByEmail(dto.email)
6     : null;
7
8   if (existing) {
9     throw new ConflictException('Email already in use');
10  }
11
12  const passwordHash = await bcrypt.hash(dto.password, 10);
13  return this.usersRepository.createOne(dto, passwordHash);
14 }
```

Listing 3: Estratto da users.service.ts

Definizione

Il **costo** di `bcrypt` (qui 10) è il numero di volte in cui l'algoritmo viene reiterato. Costi più alti rendono più lento il calcolo dell'hash, scoraggiando attacchi a forza bruta. Il valore 10 è un buon compromesso tra sicurezza e prestazioni.

3.4 Il repository: persistenza dell'hash

```
1 async createOne(dto: CreateUserDto, passwordHash: string):
2   Promise<UserEntity> {
3   const user = this.repository.create({
4     name: dto.name,
```



```
4     email: dto.email,
5     passwordHash,
6     role: dto.role,
7   });
8   return this.repository.save(user);
9 }
```

Listing 4: Estratto da `users.repository.ts`

4 La libreria @server/auth

È il cuore dell'autenticazione. Vediamone i componenti uno per volta, seguendo il flusso reale che una richiesta HTTP attraversa.

4.1 Le interfacce

Iniziamo con tre tipi che attraversano tutto il modulo.

```
1 import { UserRole } from '@server/users';
2
3 export interface AuthenticatedUser {
4   id: number;
5   email: string;
6   role: UserRole;
7   name: string;
8 }
```

Listing 5: interfaces/authenticated-user.interface.ts

```
1 import { UserRole } from '@server/users';
2
3 export interface JwtPayload {
4   sub: number; // subject = id utente (convenzione JWT)
5   email: string;
6   role: UserRole;
7   name: string;
8 }
```

Listing 6: interfaces/jwt-payload.interface.ts

```
1 import { AuthenticatedUser } from '../authenticated-user.interface';
2
3 export class AuthResponse {
4   access_token: string;
5   user: AuthenticatedUser;
6 }
```

Listing 7: interfaces/auth-response.interface.ts

lightbulb Nota

AuthenticatedUser è la versione *depurata* dell'utente: contiene tutto tranne la passwordHash. È quello che viaggia in request.user dopo che la richiesta supera una guard.

4.2 Il DTO di registrazione

Per non duplicare codice, il DTO della registrazione *estende* quello degli utenti:

```
1 import { CreateUserDto } from '@server/users';
2
3 export class RegisterDto extends CreateUserDto {}
```

Listing 8: dto/register.dto.ts

4.3 Il servizio ServerAuthService

È l'unico componente che orchestra registrazione, validazione credenziali e firma del token.

```
1 import { Injectable, NotFoundException,
2       UnauthorizedException } from '@nestjs/common';
3 import { JwtService } from '@nestjs/jwt';
4 import * as bcrypt from 'bcrypt';
5 import { ServerUsersService } from '@server/users';
6 import { AuthenticatedUser } from '../interfaces/authenticated-user.interface';
7 import { AuthResponse } from '../interfaces/auth-response.interface';
8 import { RegisterDto } from '../dto/register.dto';
9
10 @Injectable()
11 export class ServerAuthService {
12   constructor(
13     private readonly usersService: ServerUsersService,
14     private readonly jwtService: JwtService,
15   ) {}
16
17   async validateUser(email: string, password: string):
18     Promise<AuthenticatedUser> {
19     const user = await this.usersService.findByEmail(email);
20     if (!user) throw new NotFoundException('Credentials not valid');
21
22     const ok = await bcrypt.compare(password, user.passwordHash);
23     if (!ok) throw new UnauthorizedException('Credentials not valid!');
24
25     const { passwordHash, ...result } = user;
26     return result;
27   }
28
29   async login(user: AuthenticatedUser): Promise<AuthResponse> {
30     const payload = {
31       sub: user.id,
32       name: user.name,
33       email: user.email,
34       role: user.role,
35     };
36     return {
37       access_token: await this.jwtService.signAsync(payload),
38       user,
39     };
40   }
41
42   async register(dto: RegisterDto): Promise<AuthResponse> {
43     const user = await this.usersService.create(dto);
44     const { passwordHash, ...result } = user;
45     return this.login(result);
46   }
47 }
```

Listing 9: auth.service.ts

Flusso operativo

Login (validazione credenziali):

1. `validateUser(email, password)` cerca l'utente per e-mail.
2. Se non esiste → `NotFoundException`.
3. `bcrypt.compare` confronta la password fornita con l'hash salvato.
4. Se non corrisponde → `UnauthorizedException`.
5. Si rimuove il campo `passwordHash` (mai esporlo) e si restituisce `AuthenticatedUser`.

Login (emissione token):

1. Si costruisce il *payload*: `{sub, name, email, role}`.
2. `jwtService.signAsync` firma il payload con `SECRET_KEY`.
3. Si restituisce `{access_token, user}`.

4.4 La strategia LocalStrategy

Definizione

Una **strategia** è una classe che estende `PassportStrategy` e implementa il metodo `validate()`. Passport richiama `validate()` con i dati estratti dalla richiesta (e-mail e password per la *local*, payload del token per la *jwt*); il metodo deve restituire l'utente o lanciare un'eccezione.

```
1 import { Injectable } from '@nestjs/common';
2 import { PassportStrategy } from '@nestjs/passport';
3 import { Strategy } from 'passport-local';
4 import { ServerAuthService } from '../auth.service';
5 import { AuthenticatedUser } from
6     '../interfaces/authenticated-user.interface';
7
8 @Injectable()
9 export class LocalStrategy extends PassportStrategy(Strategy) {
10     constructor(private authService: ServerAuthService) {
11         super({ usernameField: 'email' }); // di default sarebbe 'username'
12     }
13
14     async validate(email: string, password: string): Promise<AuthenticatedUser>
15     {
16         return this.authService.validateUser(email, password);
17     }
18 }
```

Listing 10: strategies/local.strategy.ts

lightbulb Nota

La riga `super({usernameField: 'email'})` è *cruciale*: senza quella, Passport cercherebbe un campo `username` nel body della richiesta e fallirebbe sempre. La nostra applicazione autentica per e-mail.

4.5 La guard LocalAuthGuard

```
1 import { Injectable } from '@nestjs/common';
2 import { AuthGuard } from '@nestjs/passport';
3
4 @Injectable()
5 export class LocalAuthGuard extends AuthGuard('local') {}
```

Listing 11: guards/local-auth.guard.ts

lightbulb Nota

Questa classe fa apparentemente *niente*, ma serve a evitare le *magic strings*. Aniché scrivere `@UseGuards(AuthGuard('local'))` ovunque (con il rischio di errori di battitura non rilevati), useremo `@UseGuards(LocalAuthGuard)`.

4.6 La strategia JwtStrategy

Questa strategia non valida credenziali, ma il *token* contenuto nell'header `Authorization: Bearer <token>`.

```
1 import { Injectable } from '@nestjs/common';
2 import { PassportStrategy } from '@nestjs/passport';
3 import { ExtractJwt, Strategy } from 'passport-jwt';
4 import { AuthenticatedUser } from
5   '../interfaces/authenticated-user.interface';
6 import { JwtPayload } from '../interfaces/jwt-payload.interface';
7
8 @Injectable()
9 export class JwtStrategy extends PassportStrategy(Strategy) {
10   constructor() {
11     const secret = process.env.SECRET_KEY;
12     if (!secret) throw new Error('SECRET_KEY is not defined');
13
14     super({
15       jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
16       ignoreExpiration: false,
17       secretOrKey: secret,
18     });
19
20     validate(payload: JwtPayload): AuthenticatedUser {
21       return {
22         id: payload.sub,
23         email: payload.email,
24         role: payload.role,
25         name: payload.name,
26       };
27     }
28   }
29 }
```

Listing 12: strategies/jwt.strategy.ts

Flusso operativo

Cosa fa la JwtStrategy a ogni richiesta protetta:

1. Estrae il token dall'header **Authorization** (formato **Bearer**).
2. Verifica la firma del token con la **SECRET_KEY**.
3. Verifica che il token non sia scaduto.
4. Decodifica il *payload*.
5. Chiama `validate(payload)`: il valore restituito viene messo in `request.user` per i prossimi handler.

Attenzione

`ignoreExpiration: false` è essenziale. Mettere `true` significherebbe accettare token scaduti, di fatto rendendo eterno qualsiasi accesso anche per utenti che dovrebbero essere fuori sessione.

4.7 Il controller `ServerAuthController`

```
1 import { Body, Controller, Post, Request, UseGuards,
2       ValidationPipe } from '@nestjs/common';
3 import { ApiBody, ApiTags } from '@nestjs/swagger';
4 import { ServerAuthService } from '../auth.service';
5 import { LocalAuthGuard } from '../guards/local-auth.guard';
6 import { RegisterDto } from '../dto/register.dto';
7 import { UserRole } from '@server/users';
8 import { AuthenticatedUser } from '../interfaces/authenticated-user.interface';
9
10 interface RequestWithUser extends Request {
11   user: AuthenticatedUser;
12 }
13
14 @ApiTags('Auth APIs')
15 @Controller('auth')
16 export class ServerAuthController {
17   constructor(private readonly serverAuthService: ServerAuthService) {}
18
19   @UseGuards(LocalAuthGuard)
20   @Post('login')
21   @ApiBody({
22     schema: {
23       type: 'object',
24       properties: {
25         email: { type: 'string', example: 'mario.rossi@unibs.it' },
26         password: { type: 'string', example: 'Password1!' },
27       },
28       required: ['email', 'password'],
29     },
30   })
31   login(@Request() req: RequestWithUser) {
32     return this.serverAuthService.login(req.user);
33   }
34
35   @Post('register')
36   @ApiBody({
37     schema: {
38       type: 'object',
```

```

39   properties: {
40     name:    { type: 'string', example: 'Mario Rossi' },
41     email:   { type: 'string', example: 'mario.rossi@unibs.it' },
42     password: { type: 'string', example: 'Password1!' },
43     role: {
44       type: 'string',
45       enum: Object.values(UserRole),
46       example: UserRole.DOCENTE,
47     },
48   },
49   required: ['name', 'email', 'password', 'role'],
50 },
51 })
52 register(@Body(ValidationPipe) dto: RegisterDto) {
53   return this.serverAuthService.register(dto);
54 }
55 }

```

Listing 13: auth.controller.ts

lightbulb Nota

Notare che la rotta POST /auth/login è preceduta da @UseGuards(LocalAuthGuard). Quando la richiesta arriva, la guard innesca LocalStrategy, che estrae e valida e-mail e password dal body. Solo se la validazione ha successo l'handler login() viene effettivamente eseguito; req.user contiene già AuthenticatedUser.

4.8 Il modulo ServerAuthModule

```

1  import { Module } from '@nestjs/common';
2  import { JwtModule } from '@nestjs/jwt';
3  import { PassportModule } from '@nestjs/passport';
4  import { ServerUsersModule } from '@server/users';
5  import { ServerAuthController } from '../auth.controller';
6  import { ServerAuthService } from '../auth.service';
7  import { LocalStrategy } from '../strategies/local.strategy';
8  import { JwtStrategy } from '../strategies/jwt.strategy';
9  import 'dotenv/config';
10
11  @Module({
12    imports: [
13      ServerUsersModule,
14      PassportModule,
15      JwtModule.register({
16        secret: process.env.SECRET_KEY,
17        signOptions: { expiresIn: '24h' },
18      }),
19    ],
20    controllers: [ServerAuthController],
21    providers: [ServerAuthService, LocalStrategy, JwtStrategy],
22    exports: [ServerAuthService],
23  })
24  export class ServerAuthModule {}

```

Listing 14: auth.module.ts

4.9 File index.ts: cosa esportare e cosa no

Solo ciò che serve all'esterno della libreria:

```
1 export * from './lib/auth.module';
2 export * from './lib/auth.service';
3 export * from './lib/dto/register.dto';
4 export * from './lib/interfaces/authenticated-user.interface';
5 export * from './lib/interfaces/jwt-payload.interface';
```

Listing 15: `libs/server/auth/src/index.ts`

Tipo	Injectable	In exports	Perché
Service	✓	✓	usato via DI in altri moduli
Repository	✓	a volte	data access
Strategy	✓	×	usata internamente da Passport
Guard	✓	×	usata via <code>@UseGuards()</code>
DTO	×	×	non è un provider, è una classe dati

5 La libreria @server/security

Questa libreria contiene gli strumenti di protezione delle rotte: *guards* e *decorators*. Non parla con il database, non gestisce token, non calcola hash: delega tutto al `JwtStrategy` di `auth` e ai metadati del controller.

5.1 La guard `JwtAuthGuard`

```
1 import { Injectable } from '@nestjs/common';
2 import { AuthGuard } from '@nestjs/passport';
3
4 @Injectable()
5 export class JwtAuthGuard extends AuthGuard('jwt') {}
```

Listing 16: `guards/jwt-auth.guard.ts`

Come per `LocalAuthGuard`, è un *wrapper* che evita la magic string `'jwt'`. A runtime, attiva la `JwtStrategy` definita in `@server/auth` e popola `request.user` con l'`AuthenticatedUser`.

5.2 La guard `RolesGuard`: il cuore dell'autorizzazione

```
1 import { CanActivate, ExecutionContext, ForbiddenException,
2       Injectable } from '@nestjs/common';
3 import { Reflector } from '@nestjs/core';
4 import { UserRole } from '@server/users';
5 import { ROLES_KEY } from '../decorators/roles.decorator';
6
7 @Injectable()
8 export class RolesGuard implements CanActivate {
9   constructor(private readonly reflector: Reflector) {}
10
11   canActivate(context: ExecutionContext): boolean {
12     const requiredRoles = this.reflector.getAllAndOverride<UserRole[]>(
13       ROLES_KEY,
14       [context.getHandler(), context.getClass()],
15     );
16
17     if (!requiredRoles || requiredRoles.length === 0) {
18       return true;
19     }
20
21     const request = context.switchToHttp().getRequest();
22     const user = request.user;
23
24     if (!user) {
25       throw new ForbiddenException('User not found in request');
26     }
27
28     if (!requiredRoles.includes(user.role)) {
29       throw new ForbiddenException('Insufficient permissions');
30     }
31
32     return true;
33   }
34 }
```

Listing 17: `guards/roles.guard.ts`

Flusso operativo

Cosa fa la RolesGuard per ogni richiesta:

1. Legge i metadati ROLES_KEY associati al metodo o alla classe del controller.
2. Se nessun ruolo è richiesto, lascia passare ($\Rightarrow$ rotta «autenticata» ma non «autorizzata per ruolo»).
3. Estrae l'utente da `request.user` (popolato dalla `JwtAuthGuard`).
4. Confronta il ruolo dell'utente con la lista richiesta.
5. Se non corrisponde lancia `ForbiddenException` (HTTP 403).

Attenzione

RolesGuard non funziona da sola. Richiede che `request.user` sia già stato popolato da una `JwtAuthGuard` eseguita *prima*. L'ordine di `@UseGuards(JwtAuthGuard, RolesGuard)` è quindi significativo.

5.3 Il decoratore @Roles()

```
1 import { SetMetadata } from '@nestjs/common';
2 import { UserRole } from '@server/users';
3
4 export const ROLES_KEY = 'roles';
5 export const Roles = (...roles: UserRole[]) => SetMetadata(ROLES_KEY, roles);
```

Listing 18: `decorators/roles.decorator.ts`

È un *decoratore-factory*: la sua chiamata produce un decoratore vero e proprio. Usa `SetMetadata` per associare la chiave `ROLES_KEY` al metodo del controller. Quel metadato verrà poi letto dalla `RolesGuard` via `Reflector`.

5.4 Il decoratore @CurrentUser()

```
1 import { createParamDecorator, ExecutionContext } from '@nestjs/common';
2
3 export const CurrentUser = createParamDecorator(
4   (_data: unknown, ctx: ExecutionContext) => {
5     const request = ctx.switchToHttp().getRequest();
6     return request.user;
7   },
8 );
```

Listing 19: `decorators/current-user.decorator.ts`

È un **parameter decorator**: si applica a un parametro di un metodo. Quando il metodo viene invocato, NestJS chiama la funzione factory, che estrae `request.user` e lo inietta nel parametro. È lo zucchero sintattico che rende elegante `getMe()`.

5.5 File `index.ts` di `security`

```
1 export * from './lib/decorators/current-user.decorator';
2 export * from './lib/decorators/roles.decorator';
3 export * from './lib/guards/jwt-auth.guard';
```

```
4 export * from './lib/guards/roles.guard';
```

Listing 20: `libs/server/security/src/index.ts`

6 Applicazione al sistema di pianificazione appelli

Tutta l'infrastruttura è pronta. Vediamo come si usa concretamente per proteggere le rotte del nostro dominio.

6.1 Il ruolo come abilitazione

Riprendiamo dall'UML: gli attori sono **DOCENTE** e **SEGRETERIA**, con compiti distinti.

Attore	Operazioni autorizzate
SEGRETERIA	Crea, modifica e cancella corsi di laurea, materie e sessioni d'esame. Configura le finestre di inserimento. Non inserisce appelli.
DOCENTE	Visualizza la pianificazione completa per la propria sessione/corso. Inserisce un appello solo per le proprie materie e solo dentro la finestra di inserimento. Modifica e cancella esclusivamente i propri appelli.

6.2 Esempio: il controller delle sessioni d'esame

Le sessioni sono configurate *solo* dalla segreteria. Questo si traduce in `@Roles(UserRole.SEGRETERIA)` su tutte le rotte di scrittura.

```

1 import { Body, Controller, Delete, Get, Param,
2       Patch, Post, UseGuards } from '@nestjs/common';
3 import { ApiBearerAuth, ApiTags } from '@nestjs/swagger';
4 import { JwtAuthGuard, Roles, RolesGuard } from '@server/security';
5 import { UserRole } from '@server/users';
6 import { ExamSessionsService } from '../exam-sessions.service';
7 import { CreateExamSessionDto } from '../dto/create-exam-session.dto';
8 import { UpdateExamSessionDto } from '../dto/update-exam-session.dto';
9
10 @ApiTags('Exam Sessions APIs')
11 @Controller('exam-sessions')
12 export class ExamSessionsController {
13   constructor(private readonly service: ExamSessionsService) {}
14
15   @Get()
16   @UseGuards(JwtAuthGuard)           // basta essere autenticati
17   @ApiBearerAuth()
18   findAll() {
19     return this.service.findAll();
20   }
21
22   @Post()
23   @UseGuards(JwtAuthGuard, RolesGuard)
24   @Roles(UserRole.SEGRETERIA)       // solo segreteria
25   @ApiBearerAuth()
26   create(@Body() dto: CreateExamSessionDto) {
27     return this.service.create(dto);
28   }
29
30   @Patch('/:id')
31   @UseGuards(JwtAuthGuard, RolesGuard)
32   @Roles(UserRole.SEGRETERIA)
33   @ApiBearerAuth()
34   update(@Param('id') id: number, @Body() dto: UpdateExamSessionDto) {
35     return this.service.update(id, dto);
36   }
37

```

```
38 @Delete('/:id')
39 @UseGuards(JwtAuthGuard, RolesGuard)
40 @Roles(UserRole.SEGRETERIA)
41 @ApiBearerAuth()
42 remove(@Param('id') id: number) {
43   return this.service.remove(id);
44 }
45 }
```

Listing 21: exam-sessions.controller.ts

6.3 Esempio: il controller degli appelli

Qui le regole sono più sofisticate, perché il vincolo «un docente modifica solo i propri appelli» richiede di confrontare l'utente loggato con il *teacher* dell'appello *nel servizio*.

```
1 import { Body, Controller, Delete, Get, Param,
2   Patch, Post, UseGuards } from '@nestjs/common';
3 import { ApiBearerAuth, ApiTags } from '@nestjs/swagger';
4 import { CurrentUser, JwtAuthGuard, Roles,
5   RolesGuard } from '@server/security';
6 import { AuthenticatedUser } from '@server/auth';
7 import { UserRole } from '@server/users';
8 import { ExamsService } from '../exams.service';
9 import { CreateExamDto } from '../dto/create-exam.dto';
10 import { UpdateExamDto } from '../dto/update-exam.dto';
11
12 @ApiTags('Exams APIs')
13 @Controller('exams')
14 @UseGuards(JwtAuthGuard) // intere classi: tutte le rotte
15   authenticate
16 @ApiBearerAuth()
17 export class ExamsController {
18   constructor(private readonly service: ExamsService) {}
19
20   @Get()
21   findAll() {
22     return this.service.findAll();
23   }
24
25   @Post()
26   @UseGuards(RolesGuard)
27   @Roles(UserRole.DOCENTE)
28   create(
29     @Body() dto: CreateExamDto,
30     @CurrentUser() user: AuthenticatedUser,
31   ) {
32     return this.service.create(dto, user);
33   }
34
35   @Patch('/:id')
36   @UseGuards(RolesGuard)
37   @Roles(UserRole.DOCENTE)
38   update(
39     @Param('id') id: number,
40     @Body() dto: UpdateExamDto,
41     @CurrentUser() user: AuthenticatedUser,
42   ) {
43     return this.service.update(id, dto, user);
44   }
45
46   @Delete('/:id')
```

```
46 @UseGuards(RolesGuard)
47 @Roles(UserRole.DOCENTE)
48 remove(
49   @Param('id') id: number,
50   @CurrentUser() user: AuthenticatedUser,
51 ) {
52   return this.service.remove(id, user);
53 }
54 }
```

Listing 22: exams.controller.ts

Applicazione al progetto «Pianificazione Appelli»

Notare l'uso di `@CurrentUser()`: è il modo elegante per passare al servizio l'identità di chi sta facendo la richiesta. Il servizio confronterà `user.id` con il `teacherId` dell'appello e lancerà `ForbiddenException` se non coincidono. Questo controllo non si può fare con i decoratori, perché dipende dai dati del database.

6.4 Il servizio: il controllo «owner»

```
1 import { ForbiddenException, Injectable,
2         NotFoundException } from '@nestjs/common';
3 import { AuthenticatedUser } from '@server/auth';
4 import { Repository } from 'typeorm';
5 import { Exam } from './exam.entity';
6
7 @Injectable()
8 export class ExamsService {
9   constructor(
10     @InjectRepository(Exam) private readonly repo: Repository<Exam>,
11   ) {}
12
13   async update(id: number, dto: UpdateExamDto, user: AuthenticatedUser):
14     Promise<Exam> {
15     const exam = await this.repo.findOne({
16       where: { id },
17       relations: ['teacher'],
18     });
19     if (!exam) throw new NotFoundException('Exam not found');
20
21     if (exam.teacher.id !== user.id) {
22       throw new ForbiddenException('Puoi modificare solo i tuoi appelli');
23     }
24
25     Object.assign(exam, dto);
26     return this.repo.save(exam);
27 }
```

Listing 23: Estratto da exams.service.ts

7 Configurazione Swagger con autenticazione

Per testare le rotte protette dalla UI di Swagger occorre dichiarare il *security scheme* Bearer in `main.ts`.

```
1 const config = new DocumentBuilder()
2   .addBearerAuth()
3   .setTitle('Pianificazione Appelli')
4   .setDescription('API DOCUMENTATION')
5   .setVersion('1.0')
6   .addTag('api')
7   .build();
```

Listing 24: `apps/api/src/main.ts`

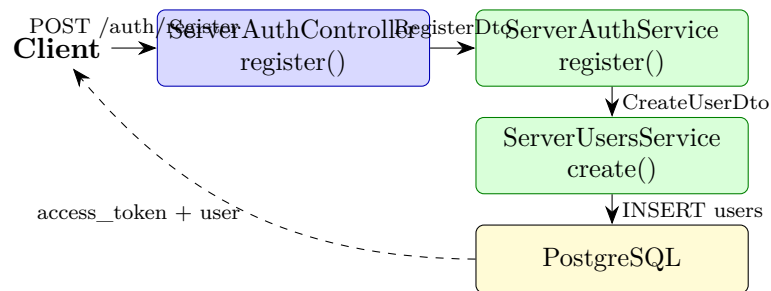
Flusso operativo

Per provare un endpoint protetto in Swagger:

1. Eseguire `POST /auth/register` (o `/auth/login`).
2. Copiare il valore di `access_token` dalla risposta.
3. Cliccare il pulsante *Authorize* in alto a destra di Swagger.
4. Incollare il token (senza il prefisso `Bearer`, lo aggiunge Swagger).
5. Da quel momento tutte le rotte segnate con il lucchetto useranno il token.

8 Flusso end-to-end di una richiesta

8.1 Registrazione di un nuovo docente



8.2 Richiesta protetta da @Roles(SEGRETERIA)

1. Il client invia `POST /api/exam-sessions` con header `Authorization: Bearer eyJ...`
2. NestJS riceve la richiesta e attiva la prima guard: `JwtAuthGuard`.
3. `JwtAuthGuard` delega a `JwtStrategy`: estrae il token, ne verifica la firma con `SECRET_KEY`, controlla la scadenza, decodifica il payload.
4. `validate(payload)` costruisce l'oggetto `AuthenticatedUser` e lo mette in `request.user`.
5. Si attiva la seconda guard: `RolesGuard`.
6. `RolesGuard` legge il metadato `ROLES_KEY (= [SEGRETERIA])`, controlla che `request.user.role` corrisponda. Se è `DOCENTE` → 403.
7. Solo se entrambe le guards permettono il passaggio, viene chiamato `ExamSessionsController.create()`.
8. Il servizio esegue la business logic, salva su DB e restituisce la risposta.

9 Esercizi di verifica

Esercizio 1 — Comprensione concettuale

Per ognuno dei seguenti scenari, indica se il problema è di autenticazione, autorizzazione o entrambe:

1. Un utente prova ad accedere a `POST /exam-sessions` senza header `Authorization`.
2. Un docente con token valido tenta `DELETE /exam-sessions/5`.
3. Un utente fornisce password sbagliata in `POST /auth/login`.
4. La segreteria tenta di modificare un appello inserito da un docente.

Esercizio 2 — Pratico

Implementa una nuova rotta `GET /exams/mine` che restituisce *solo* gli appelli del docente loggato. Vincoli:

- accessibile solo a utenti con ruolo `DOCENTE`;
- usa `@CurrentUser()` per estrarre l'utente;
- nel servizio filtra per `teacher.id = user.id`.

Esercizio 3 — Sicurezza

Cosa succederebbe se nel `ServerAuthService` si dimenticasse il `const { passwordHash, ...result } = user;` prima del `return result;`? Quale dato sensibile finirebbe nel JWT e nelle risposte API?

Esercizio 4 — Estensione

Aggiungi un terzo ruolo `ADMIN` con i permessi della segreteria *più* la possibilità di cancellare qualunque appello (anche di altri docenti). Quali file modifichi? Quali decorator aggiungi e dove?

10 Riepilogo finale

Mappa mentale

Tre librerie, tre responsabilità:

- `@server/users` → chi sono gli utenti, come sono memorizzati
- `@server/auth` → come si autenticano, come si emette il token
- `@server/security` → come si proteggono le rotte

Quattro componenti chiave:

- *Strategy* → algoritmo di verifica (local, jwt)
- *Guard* → filtro di accesso a una rotta
- *Decorator* → metadato letto dalla guard, oppure injection di dati
- *Service* → business logic, hashing, firma token

Due flussi distinti:

- *Login*: `LocalAuthGuard` → `LocalStrategy` → `validateUser` → `login` → token
- *Richiesta protetta*: `JwtAuthGuard` → `JwtStrategy` → `request.user` → `RolesGuard` → handler

Riferimenti

- Documentazione ufficiale: <https://docs.nestjs.com/recipes/passport>
- JSON Web Tokens: <https://jwt.io>
- Bcrypt: <https://github.com/kelektiv/node.bcrypt.js>
- Class-validator: <https://github.com/typestack/class-validator>